

VPS Deployment via Docker Compose

This guide covers how to deploy the Webmap application onto a VPS utilizing Docker and Docker Compose. This is the **recommended** approach, as it isolates dependencies and simplifies teardown and updates.

Prerequisites

- A VPS (Ubuntu/Debian recommended).
 - A registered domain name (e.g., webmap.rhdbridge.com) pointed to your VPS IP address.
-

1. Install Docker and Git

Log into your VPS and install Docker and Docker Compose:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git docker.io docker-compose
sudo systemctl enable --now docker
```

2. Clone the Repository

Clone your project repository onto the server:

```
git clone https://your-repo-url/webmap.git
cd webmap
```

3. Create the Secrets File

Docker Compose relies on a `.env` file located in the exact same directory as your `docker-compose.yml`. This file securely holds your secrets.

Create the file:

```
nano .env
```

Paste in your environment variables. Make sure you set a secure Webmaster username and password. (Note: Replace `<webmap_db>`, `<webmap_user>`, and the passwords with your actual desired names/passwords):

```
# Webmaster Dashboard Credentials
WEBMASTER_USERNAME=<your_secure_admin_name>
WEBMASTER_PASSWORD=<your_super_secret_password>
```

```
# Database Credentials
POSTGRES_USER=<webmap_user>
POSTGRES_PASSWORD=<your_secure_password>
POSTGRES_DB=<webmap_db>
```

Save and exit (**Ctrl+O**, **Enter**, **Ctrl+X**).

4. Boot up the Containers

Since all of the configurations are already defined in the `docker-compose.yml` and `Dockerfile`, deployment is a single command.

Run Docker Compose in detached mode:

```
sudo docker-compose up -d --build
```

What happens next?

1. Docker pulls the official PostGIS image and starts the database container (`db`).
 2. Docker builds your Python backend container, resolving dependencies via `uv.lock`.
 3. The FastAPI backend boots up and binds to the network.
 4. An automated background task in FastAPI silently executes `import_local_maps.py` to ingest the GeoJSON vector layers into the PostGIS database.
 5. The `frontend` Nginx container starts, serving your HTML/JS and proxying `/api` requests to the backend.
-

5. Verify the Deployment (HTTP Protocol)

You can monitor the automated background map ingestion process by viewing the backend logs:

```
sudo docker-compose logs -f backend
```

Look for `Background data ingestion completed with code 0`. Once you see this, your Vector Layers are successfully loaded into PostGIS.

Your application is now running natively inside Docker on HTTP (Port 80). If you navigate to `http://your-vps-ip`, the map will be fully operational over standard HTTP.

6. Secure with SSL (HTTPS Protocol)

To serve the application securely over `https://webmap.rhdbridge.com`, you must route the traffic through a Reverse Proxy on your host machine to handle the SSL decryption before passing it to Docker.

6.1 Install Nginx on the Host VPS

Install Nginx and Certbot on your host machine (outside of Docker):

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

6.2 Configure Host Nginx to Proxy to Docker

Create a configuration file:

```
sudo nano /etc/nginx/sites-available/webmap
```

Paste this HTTP configuration, which simply forwards traffic from `webmap.rhdbridge.com` into your Docker container running on `127.0.0.1:80`:

```
server {  
    listen 80;  
    server_name webmap.rhdbridge.com;  
  
    location / {  
        proxy_pass http://127.0.0.1:80;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Enable the configuration and test it:

```
sudo ln -s /etc/nginx/sites-available/webmap /etc/nginx/sites-enabled/  
sudo nginx -t  
sudo systemctl restart nginx
```

6.3 Enable HTTPS (Two Methods)

Method A: Automated SSL with Certbot (Recommended) Run Certbot to automatically fetch free Let's Encrypt certificates and modify your Nginx block to handle HTTPS (Port 443):

```
sudo certbot --nginx -d webmap.rhdbridge.com
```

Method B: Manual SSL Configuration If your domain manager provides custom certificates (`.crt` and `.key`), upload them to `/etc/ssl/certs/` and `/etc/ssl/private/`. Then, update `/etc/nginx/sites-available/webmap`:

```
# Redirect HTTP to HTTPS
server {
    listen 80;
    server_name webmap.rhdbridge.com;
    return 301 https://$host$request_uri;
}

# Serve HTTPS
server {
    listen 443 ssl;
    server_name webmap.rhdbridge.com;

    ssl_certificate /etc/ssl/certs/your_domain.crt;
    ssl_certificate_key /etc/ssl/private/your_domain.key;

    location / {
        proxy_pass http://127.0.0.1:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Restart Nginx (`sudo systemctl restart nginx`). Your Dockerized Webmap is now fully secured via HTTPS!